

Di seguito trovi un **mini-progetto Spring Boot completo e già strutturato** con:

✓ Entity ✓ Repository ✓ Service ✓ Controller REST ✓ DTO ✓ Mapper ✓ gestione CRUD ✓ gestione errori base (404)

Tema: **Gestione Studenti.**



STRUTTURA PROGETTO (consigliata)

```
src/main/java/com/example/scuola
├── controller
│   └── StudenteController.java
├── dto
│   └── StudenteDTO.java
├── entity
│   └── Studente.java
├── mapper
│   └── StudenteMapper.java
├── repository
│   └── StudenteRepository.java
├── service
│   └── StudenteService.java
├── exception
│   └── ResourceNotFoundException.java
└── ScuolaApplication.java
```



1) ENTITY

entity/Studente.java

```
package com.example.scuola.entity;

import jakarta.persistence.*;

@Entity
@Table(name = "studenti")
public class Studente {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String nome;
    private String cognome;
    private String classe;
```

```

private double media;

public Studente() {}

public Studente(Long id, String nome, String cognome, String classe,
double media) {
    this.id = id;
    this.nome = nome;
    this.cognome = cognome;
    this.classe = classe;
    this.media = media;
}

public Long getId() { return id; }
public void setId(Long id) { this.id = id; }

public String getName() { return nome; }
public void setName(String nome) { this.nome = nome; }

public String getCognome() { return cognome; }
public void setCognome(String cognome) { this.cognome = cognome; }

public String getClasse() { return classe; }
public void setClasse(String classe) { this.classe = classe; }

public double getMedia() { return media; }
public void setMedia(double media) { this.media = media; }
}

```

✓ 2) DTO

📌 dto/StudenteDTO.java

```

package com.example.scuola.dto;

public class StudenteDTO {

    private Long id;
    private String nome;
    private String cognome;
    private String classe;
    private double media;

    public StudenteDTO() {}

    public StudenteDTO(Long id, String nome, String cognome, String classe,
double media) {
        this.id = id;
        this.nome = nome;

```

```

        this.cognome = cognome;
        this.classe = classe;
        this.media = media;
    }

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getNome() { return nome; }
    public void setNome(String nome) { this.nome = nome; }

    public String getCognome() { return cognome; }
    public void setCognome(String cognome) { this.cognome = cognome; }

    public String getClasse() { return classe; }
    public void setClasse(String classe) { this.classe = classe; }

    public double getMedia() { return media; }
    public void setMedia(double media) { this.media = media; }
}

```

✓ 3) MAPPER (DTO ↔ ENTITY)

📍 mapper/StudenteMapper.java

```

package com.example.scuola.mapper;

import com.example.scuola.dto.StudenteDTO;
import com.example.scuola.entity.Studente;

public class StudenteMapper {

    public static StudenteDTO toDTO(Studente s) {
        return new StudenteDTO(
            s.getId(),
            s.getNome(),
            s.getCognome(),
            s.getClasse(),
            s.getMedia()
        );
    }

    public static Studente toEntity(StudenteDTO dto) {
        return new Studente(
            dto.getId(),
            dto.getNome(),
            dto.getCognome(),
            dto.getClasse(),
            dto.getMedia()
        );
    }
}

```

```
        );  
    }  
  
    public static void updateEntityFromDTO(StudenteDTO dto, Studente  
entity) {  
        entity.setNome(dto.getNome());  
        entity.setCognome(dto.getCognome());  
        entity.setClasse(dto.getClasse());  
        entity.setMedia(dto.getMedia());  
    }  
}
```

✓ 4) REPOSITORY

📌 repository/StudenteRepository.java

```
package com.example.scuola.repository;  
  
import com.example.scuola.entity.Studente;  
import org.springframework.data.jpa.repository.JpaRepository;  
import java.util.List;  
  
public interface StudenteRepository extends JpaRepository<Studente, Long> {  
  
    List<Studente> findByClasse(String classe);  
}
```

✓ 5) ECCEZIONE PERSONALIZZATA (404)

📌 exception/ResourceNotFoundException.java

```
package com.example.scuola.exception;  
  
public class ResourceNotFoundException extends RuntimeException {  
  
    public ResourceNotFoundException(String message) {  
        super(message);  
    }  
}
```

✓ 6) SERVICE

```
package com.example.scuola.service;

import com.example.scuola.dto.StudenteDTO;
import com.example.scuola.entity.Studente;
import com.example.scuola.exception.ResourceNotFoundException;
import com.example.scuola.mapper.StudenteMapper;
import com.example.scuola.repository.StudenteRepository;
import org.springframework.stereotype.Service;

import java.util.List;

@Service
public class StudenteService {

    private final StudenteRepository repository;

    public StudenteService(StudenteRepository repository) {
        this.repository = repository;
    }

    public List<StudenteDTO> getAll() {
        return repository.findAll()
            .stream()
            .map(StudenteMapper::toDTO)
            .toList();
    }

    public StudenteDTO getById(Long id) {
        Studente s = repository.findById(id)
            .orElseThrow(() -> new ResourceNotFoundException("Studente non trovato con id=" + id));

        return StudenteMapper.toDTO(s);
    }

    public List<StudenteDTO> getByClasse(String classe) {
        return repository.findByClasse(classe)
            .stream()
            .map(StudenteMapper::toDTO)
            .toList();
    }

    public StudenteDTO create(StudenteDTO dto) {
        Studente entity = StudenteMapper.toEntity(dto);
        entity.setId(null); // forzo ID generato dal DB

        Studente salvato = repository.save(entity);
        return StudenteMapper.toDTO(salvato);
    }

    public StudenteDTO update(Long id, StudenteDTO dto) {
```

```

        Studente entity = repository.findById(id)
            .orElseThrow(() -> new ResourceNotFoundException("Studente
non trovato con id=" + id));

        StudenteMapper.updateEntityFromDTO(dto, entity);

        Studente salvato = repository.save(entity);
        return StudenteMapper.toDTO(salvato);
    }

    public void delete(Long id) {
        Studente entity = repository.findById(id)
            .orElseThrow(() -> new ResourceNotFoundException("Studente
non trovato con id=" + id));

        repository.delete(entity);
    }
}

```

✓ 7) CONTROLLER REST

📌 controller/StudenteController.java

```

package com.example.scuola.controller;

import com.example.scuola.dto.StudenteDTO;
import com.example.scuola.service.StudenteService;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/studenti")
public class StudenteController {

    private final StudenteService service;

    public StudenteController(StudenteService service) {
        this.service = service;
    }

    @GetMapping
    public ResponseEntity<List<StudenteDTO>> getAll() {
        return ResponseEntity.ok(service.getAll());
    }

    @GetMapping("/{id}")
    public ResponseEntity<StudenteDTO> getById(@PathVariable Long id) {

```

```

        return ResponseEntity.ok(service.getById(id));
    }

    @GetMapping("/classe/{classe}")
    public ResponseEntity<List<StudenteDTO>> getByClasse(@PathVariable
String classe) {
        return ResponseEntity.ok(service.getByClasse(classe));
    }

    @PostMapping
    public ResponseEntity<StudenteDTO> create(@RequestBody StudenteDTO dto)
{
        return ResponseEntity.ok(service.create(dto));
    }

    @PutMapping("/{id}")
    public ResponseEntity<StudenteDTO> update(@PathVariable Long id,
@RequestBody StudenteDTO dto) {
        return ResponseEntity.ok(service.update(id, dto));
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable Long id) {
        service.delete(id);
        return ResponseEntity.noContent().build();
    }
}

```

✓ 8) GESTIONE ERRORI (OPZIONALE MA CONSIGLIATA)

📌 controller/GlobalExceptionHandler.java

```

package com.example.scuola.controller;

import com.example.scuola.exception.ResourceNotFoundException;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<String> handleNotFound(ResourceNotFoundException
ex) {
        return ResponseEntity.status(404).body(ex.getMessage());
    }
}

```

```
}  
}
```

✓ 9) APPLICATION.PROPERTIES (H2 DATABASE)

📌 `src/main/resources/application.properties`

```
spring.datasource.url=jdbc:h2:mem:scuola  
spring.datasource.driverClassName=org.h2.Driver  
spring.datasource.username=sa  
spring.datasource.password=  
  
spring.jpa.hibernate.ddl-auto=update  
spring.jpa.show-sql=true  
  
spring.h2.console.enabled=true  
spring.h2.console.path=/h2-console
```

✓ 10) MAIN

📌 `ScuolaApplication.java`

```
package com.example.scuola;  
  
import org.springframework.boot.SpringApplication;  
import org.springframework.boot.autoconfigure.SpringBootApplication;  
  
@SpringBootApplication  
public class ScuolaApplication {  
  
    public static void main(String[] args) {  
        SpringApplication.run(ScuolaApplication.class, args);  
    }  
}
```

🧪 TEST API (Postman / Curl)

Creazione studente (POST)

POST `http://localhost:8080/api/studenti`

Body JSON:

```
{
  "nome": "Mario",
  "cognome": "Rossi",
  "classe": "5A",
  "media": 7.5
}
```

Tutti gli studenti (GET)

GET `http://localhost:8080/api/studenti`

Studente per id

GET `http://localhost:8080/api/studenti/1`

Studenti per classe

GET `http://localhost:8080/api/studenti/classe/5A`

Update studente

PUT `http://localhost:8080/api/studenti/1`

```
{
  "nome": "Mario",
  "cognome": "Rossi",
  "classe": "5A",
  "media": 8.0
}
```

Delete studente

DELETE `http://localhost:8080/api/studenti/1`



DIPENDENZE MAVEN (pom.xml)

```
<dependencies>
```

```
  <dependency>
```

```
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>

<dependency>
<groupId>com.h2database</groupId>
<artifactId>h2</artifactId>
<scope>runtime</scope>
</dependency>

</dependencies>
```

RISULTATO FINALE

Hai un progetto con architettura pulita:

- Controller → espone API REST
- Service → logica applicativa
- Repository → accesso DB con JPA
- Entity → tabella DB
- DTO → oggetto di trasporto
- Mapper → conversione DTO ↔ Entity